

Module 7: Lesson 1

# Introduction to Neural Networks



# Outline

- ▶ Road map of the module
- ▶ Neural networks
- ▶ Linear regression as a neural network

# Road map for the module

The final module of ML serves as an introduction to your next course on Deep Learning for Finance. Thus, we will start by revisiting some of the classic concepts (e.g., linear regression) using deep learning vocabulary. It will also help in introducing the environment we will be working with in the next course: Tensorflow.

- ▶ **Lesson 1** revisits linear regression as a neural network, introducing some important concepts that we will use later on (e.g., minibatch stochastic gradient descent, ...)
- ▶ **Lesson 2** introduces the Tensorflow environment on a practical linear regression setting: we will test if linear models have any market timing ability by evaluating smart beta trading strategies.
- ▶ **Lesson 3** introduces Multi-Layer Perceptron (MLP) networks.
- ▶ **Lesson 4** tackles the predictability problem of smart beta trading strategies using an MLP neural network to compare the performance of the algorithms with respect to the classic linear case.

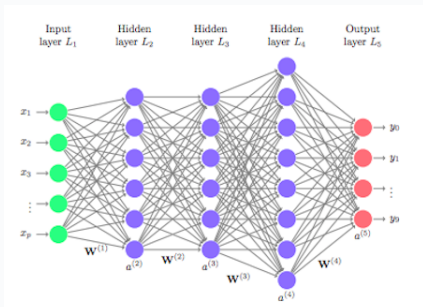
So, let's start by looking at linear regression, something that you are already very familiar with, through the lens of a neural network.

# Neural Networks

The final module of ML serves as an introduction to your next course on Deep Learning for Finance. Thus, we will start by revisiting some of the classic concepts (e.g., linear regression) using neural network vocabulary.

- The theoretical development of this lesson is on **Chapter 3** from the book by [Zhang, Aston, et al. \*Dive into Deep Learning\*. 2022.](#)

A **neural network (NN)** can simply be seen as a series of different algorithms that aim to recognize patterns in the input data by processing information through different layers and neurons:



# Linear Regression as a Neural Network

Before dealing with deep neural networks, it will be useful to think about a linear regression problem as a single-layer neural network.

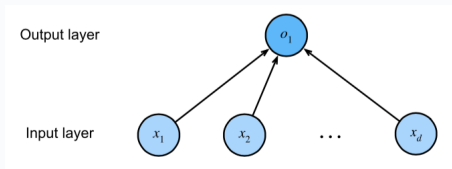
- We know already the classic linear regression equation form:

$$y = Xw + b$$

where  $w$  that minimize a squared error (loss) function can be expressed in closed-form as:

$$w^* = (X^T X)^{-1} X^T y$$

- Now, linear regression can be expressed as a single-layer neural network:



Let's see how this works.

# Linear Regression as a Neural Network

There are 4 main features that will characterize a simple neural network model such as the one for linear regression:

1. The model
2. The loss function
3. The optimization algorithm
4. The training function

⇒ In constructing a model, we will have to make choices to define each of these features.

Most of the time, as we work with libraries like Tensorflow, defining all these features would be just a few lines of code. However, it is crucial that we understand what is going on behind the scenes because this will allow us to:

- ▶ Make better choices depending on the tasks at hand.
- ▶ Identify where problems in model performance stem from.

Next, we review these different features...

# Linear Regression as a Neural Network

## 1. The model

In the case of linear regression, the model is quite simple, and it simply encompasses 1 layer of neurons (each corresponding to a different input) that are combined to form the output in the following linear way:

$$y = Xw + b$$

that is, going **forward** from the input layer to the output layer is done by a matrix-vector product of input features  $X$  and weights  $w$  and then adding the scalar  $b$  (bias term) using broadcasting.

→ Of course, we are aware that, in the linear case, there is a closed-form solution to the form of weights ( $w$ ) that minimizes the squared error.

→ However, we are going to use neural network features relying on iterations to obtain the solution to the problem:

- ▶ How do we establish the initial values for the weights ( $w$ )?
  - Matrix/vector of zeros? → No good
  - Random numbers? → Yes, better! (how?)
- ▶ How do we establish initial values for bias term ( $b$ )? Zero!

# Linear Regression as a Neural Network

## 2. The loss function

We need a loss function to use in our optimization problem with neural networks. This loss function will measure the fitness of the model (or better said, unfitness).

It is common, especially in the linear case, to use a squared error loss function, defined such that for each example  $i$ :

$$\ell^{(i)}(w, b) = \frac{1}{2} \left( \hat{y}^{(i)} - y^{(i)} \right)^2$$

Note that the  $1/2$  is just a convenient way to express it to cancel out the power in the derivative.

If we want to measure the fit of the whole model we simply average on  $n$  training set examples:

$$L(w, b) = \frac{1}{n} \sum_{i=1}^n \ell^{(i)}(w, b) = \frac{1}{n} \sum_{i=1}^n \frac{1}{2} \left( w^T x^{(i)} + b - y^{(i)} \right)^2$$



# Linear Regression as a Neural Network

## 3. The optimization algorithm

This is one of the most important pieces to understand in the development of a model. First, it is important we understand the main optimization technique most algorithms rely on: **stochastic gradient descent (SGD)**

We already know how gradient descent works. **SGD** reduces computational costs because the gradient is computed from a uniform random sample of  $i$  examples in the data. Thus, we update  $\mathbf{x}$ , using a learning rate  $\eta$  as:

$$\mathbf{x} \leftarrow \mathbf{x} - \eta \nabla f_i(\mathbf{x})$$

Moreover, to increase speed we are going to use **minibatch SGD**:

- ▶ When updating the SGD algorithm, every observation takes time. Parsing the entire dataset for every update of the algorithm does as well.
- ▶ So, a faster intermediate strategy is taking a **minibatch** of observations, where weights are updated as:

$$\mathbf{w} \leftarrow \mathbf{w} - \frac{\eta}{|\mathcal{B}|} \sum_{i \in \mathcal{B}_t} \partial_{\mathbf{w}} \ell^{(i)}(\mathbf{w}, \mathbf{b})$$

# Linear Regression as a Neural Network

## 4. The training

Training is the moment of the DL algorithm where we put all the other parts together. As we have already mentioned, the training of a DL algorithm is aimed at minimizing the loss function with an iterative process:

- ▶ In each **iteration**, we take a minibatch of examples to compute gradients and update model parameters.
- ▶ In each **epoch**, we iterate through the entire training set.

This is essentially a loop. What is the structure of the training process?

1. Initialize parameters  $\rightarrow (w, b)$

2. Loop **until done**:

- ▶ Compute gradients:  $g \leftarrow \partial_{(w,b)} \frac{1}{|\mathcal{B}|} \sum_{i \in \mathcal{B}} \ell(x^{(i)}, y^{(i)}, w, b)$
- ▶ Update parameters:  $(w, b) \leftarrow (w, b) - \eta g$

$\Rightarrow$  **What does "until done" mean?** Stop when it reaches a pre-specified condition (e.g., max epochs, min error, etc.)

# Summary of Lesson 1

In Lesson 1, we have looked at:

- ▶ Basics of Neural Networks (NNs)
- ▶ Linear regression as an NN

⇒ **References for this lesson:**

Chapters 3 and 12.4 in [Zhang, Aston, et al. \*Dive into Deep Learning\*. 2022.](#)

**TO DO NEXT:** This lesson does not have an associated Jupyter Notebook. Please advance to the next lesson.

In the next lesson, we will implement linear regression as a neural network using Tensorflow in a finance context application.